



Bikkad IT Institute

19. SQL Common Table Expressions (CTE)

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

SQL Common Table Expressions

19 · CTEs — Complete Reference Guide

Table of Contents

1. Non-Recursive CTE
2. Recursive CTE — Generate Sequence
3. Recursive CTE — Build Hierarchy

1. Non-Recursive CTE

→ *Step 1: Total sales per customer*

```
WITH CTE_Total_Sales AS (  
    SELECT  
        CustomerID,  
        SUM(Sales) AS TotalSales  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)
```

→ *Step 2: Last order date per customer*

```
, CTE_Last_Order AS (  
    SELECT  
        CustomerID,  
        MAX(OrderDate) AS Last_Order  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)
```

→ *Step 3: Rank customers by total sales (Nested CTE referencing Step 1)*

```
, CTE_Customer_Rank AS (  
    SELECT  
        CustomerID,  
        TotalSales,  
        RANK() OVER (ORDER BY TotalSales DESC) AS CustomerRank  
    FROM CTE_Total_Sales
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajnagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

)

→ *Step 4: Segment customers — High / Medium / Low (Nested CTE referencing Step 1)*

```
, CTE_Customer_Segments AS (
    SELECT
        CustomerID,
        TotalSales,
        CASE
            WHEN TotalSales > 100 THEN 'High'
            WHEN TotalSales > 80  THEN 'Medium'
            ELSE 'Low'
        END AS CustomerSegments
    FROM CTE_Total_Sales
)
```

→ *Main Query: Combine all CTEs*

```
SELECT
    c.CustomerID, c.FirstName, c.LastName,
    cts.TotalSales,
    clo.Last_Order,
    ccr.CustomerRank,
    ccs.CustomerSegments
FROM Sales.Customers AS c
LEFT JOIN CTE_Total_Sales      AS cts ON cts.CustomerID = c.CustomerID
LEFT JOIN CTE_Last_Order      AS clo ON clo.CustomerID = c.CustomerID
LEFT JOIN CTE_Customer_Rank   AS ccr ON ccr.CustomerID = c.CustomerID
LEFT JOIN CTE_Customer_Segments AS ccs ON ccs.CustomerID = c.CustomerID;
```

2. Recursive CTE — Generate Sequence

→ *Task 2: Generate a sequence of numbers from 1 to 20*

```
WITH Series AS (
    -- Anchor Query
    SELECT 1 AS MyNumber
    UNION ALL
    -- Recursive Query
    SELECT MyNumber + 1
    FROM Series
    WHERE MyNumber < 20
)
SELECT * FROM Series
```

→ *Task 3: Generate a sequence from 1 to 1000 (increase MAXRECURSION)*

```
WITH Series AS (
    -- Anchor Query
    SELECT 1 AS MyNumber
    UNION ALL
```

OUR LOCATIONS

```
-- Recursive Query
SELECT MyNumber + 1
FROM Series
WHERE MyNumber < 1000
)
SELECT * FROM Series
OPTION (MAXRECURSION 5000);
```

3. Recursive CTE — Build Hierarchy

→ *Task 4: Employee hierarchy — each employee's level in the organisation*

```
WITH CTE_Emp_Hierarchy AS (
  -- Anchor Query: top-level employees (no manager)
  SELECT
    EmployeeID,
    FirstName,
    ManagerID,
    1 AS Level
  FROM Sales.Employees
  WHERE ManagerID IS NULL

  UNION ALL

  -- Recursive Query: subordinates, increment level
  SELECT
    e.EmployeeID,
    e.FirstName,
    e.ManagerID,
    Level + 1
  FROM Sales.Employees AS e
  INNER JOIN CTE_Emp_Hierarchy AS ceh
    ON e.ManagerID = ceh.EmployeeID
)
SELECT * FROM CTE_Emp_Hierarchy;
```